Assignment 1: Basic Operations, Edge Extraction by Convolution, And Histogram Equalization

Piper Inns Hall
COMP361
Victoria University of Wellington
Wellington, New Zealand
innshpipe@myvuw.ac.nz

I. Introduction

This program implements basic image processing operations using the OpenCV library, written in Rust. It covers color space conversion and channel manipulation, edge extraction via manual convolution, and histogram equalization.

The program requires Rust and OpenCV to be installed. Before running, extract `assets.zip` to create the `assets/` directory. Output images will be written to the `assets/` directory. Running this program will call the Core, Completion, and Extension Functions.

```
unzip assets.zip
cargo run
```

II. Core

`fn convert_matrix_color_space()`

Converts a matrix to a different color space using OpenCV's `cvt_color` function, taking the source image matrix and an OpenCV color space conversion code (e.g. `COLOR_BGR2HSV`).

`fn split_channels()`

Splits a multi-channel matrix into a vector of single-channel matrices.

`fn multiply_channel()`

Scales each HSV channel independently by 0.0, 0.2, 0.4, 0.6, and 0.8, returning three vectors of five matrices each: one vector per channel.

`fn create_large_image()`

Arranges a vector of same-sized matrices into a single grid image, taking the number of rows, columns, and output matrix type as parameters.

`fn euclidean_mask()`

Creates a binary mask by computing the Euclidean distance of every pixel to the pixel at (80, 80). Pixels within a distance of 100 are set to 255, all others to 0.

A. Core 1

Fig. 1. BGR and HSV channels

Fig. 1 shows the Six channels of `Flower.jpg` split across 2 rows. The top row shows the separate B, G, and R channels. The R channels is the brightest channel, showing the flowers orange/yellow coloring. The background of the G channel is light, reflecting the grass behind the flower. The B channel is the darkest channel here.

The bottom row of Fig. 1 shows the H, S, and V color channels. The Hue appears nearly black because orange and yellow hues sit in the low end of the hue range (roughly 0-60°), giving them small values that appear darker in the greyscale image. The S channel is bright where colours are vivid. The flower petals appear white since they are very saturated, while more neutral areas like the background grass appear darker gray. The V channel closely resembles a standard greyscale image, as it captures the brightness of each pixel independent of its colour.

B. Core 2

Fig. 2. HSV channel scaling

Fig. 2 shows the effect of scaling each HSV channel independently by 0.0, 0.2, 0.4, 0.6, and 0.8. Scaling H shifts the hue toward red at lower values. Scaling S removes colour information, producing a grey image at 0.0. Scaling V reduces brightness, producing a black image at 0.0.

C. Core 3

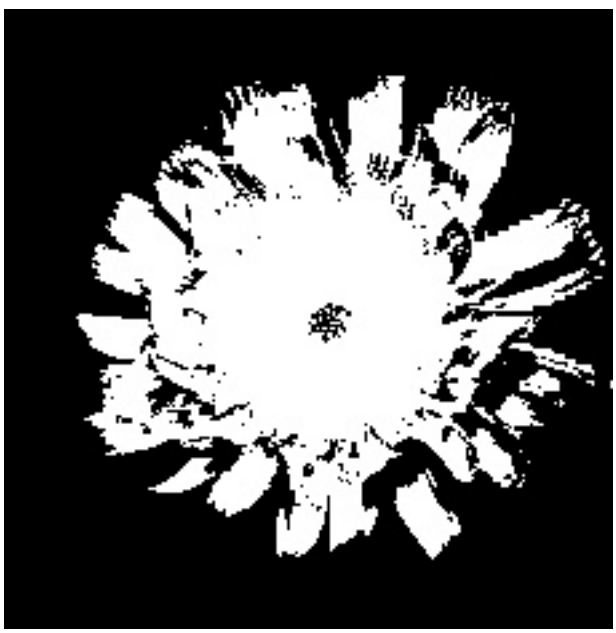

Fig. 3. Euclidean distance mask

Fig. 3 shows a binary mask where white pixels are within a Euclidean distance of 100 from the pixel at (80, 80) in RGB colour space, and black pixels are not.

In this case, Euclidean distance uses the Pythagorean theorem to measure the straight-line distance between two pixels in RGB colour space, computed as $\sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}$

III. Completion

```
fn edge_detection()
```

Applies Laplacian, Sobel-x, and Sobel-y kernels to a greyscale matrix via manual convolution, returning three normalized edge images.

The kernels are defined as:

```
let laplacian = [[0.0,  1.0, 0.0],
                 [1.0, -4.0, 1.0],
                 [0.0,  1.0, 0.0]];

let sobel_x = [[-1.0, 0.0, 1.0],
               [-2.0, 0.0, 2.0],
               [-1.0, 0.0, 1.0]];

let sobel_y = [[-1.0, -2.0, -1.0],
               [ 0.0,  0.0,  0.0],
               [ 1.0,  2.0,  1.0]];
```

The matrix is padded with a reflective border before convolution to prevent edge artifacts. Each pixel's 3 by 3 neighborhood is extracted and the dot product with the kernel is computed. The result is normalized so that the most negative value maps to 0, zero maps to 127, and the most positive value maps to 255.

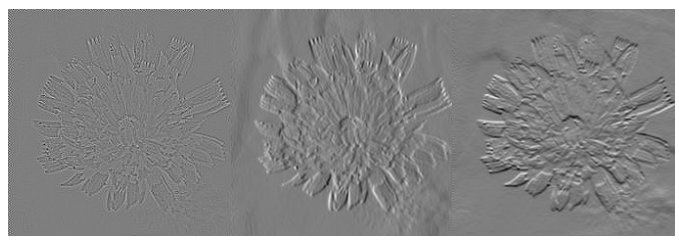

Fig. 4. Laplacian Fig. 5. Sobel X Fig. 6. Sobel Y

Fig. 4 shows that Laplacian detects edges in all directions, producing a flat, uniform response around edges. In Fig. 5 the Sobel X response is strongest on vertical edges, appearing spread horizontally. The Sobel Y response is strongest on horizontal edges, and Fig. 6 appears spread vertically.

IV. Challenge

```
fn convert_histogram()
```

Computes a 256-bin histogram from a single-channel greyscale matrix using OpenCV's built in `calc_hist`.

```
fn equalize_histogram()
```

Equalizes a greyscale image by remapping pixel intensities via a lookup table.

The PDF is computed by dividing each bin count by the total number of pixels. The CDF is the running sum of the

PDF, scaled to [0, 255] to produce the lookup table. Each pixel intensity is then remapped using the lookup table.

Fig. 7. Histogram equalization result

Fig. 7 shows the equalized `Building.jpg`. The output has stronger contrast, with shadows appearing darker and highlights brighter compared to the original.

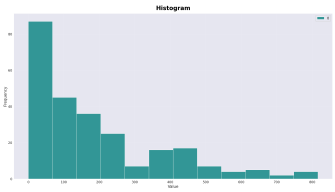

Fig. 8. before

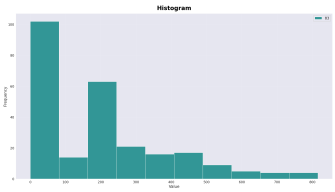

Fig. 9. after

Fig. 8 shows the original histogram, with pixel intensities clustered in a narrow range. Fig. 9 shows the equalized histogram, with intensities redistributed across the full [0, 255] range, with a concentration toward the brighter end.

V. AI Disclosure

Claude (Anthropic) was used to assist with Typst (LaTeX like writing tool) syntax and selecting code snippets for the report. No AI tools were used in writing the program.